

setup

overhead

tag

```
In[28]:= home = "projects/bae/hyperion/simulation/confidence/";
Get["utility modules.m", Path → dirPack];
Needs["DatabaseLink`"];
stamp1;

... CreateDirectory: /home/dantopa/Mathematica_files/io/ already exists.
... CreateDirectory: /mnt/T7-Shield/Dropbox/_mm/io/projects/ already exists.
... CreateDirectory: /mnt/T7-Shield/Dropbox/_mm/io/projects/bae/ already exists.
... General: Further output of CreateDirectory::eexist will be suppressed during this calculation. ⓘ

maximum memory: 0.165271 GB
seed file: /home/dantopa/Mathematica_files/nb/seed 23_01.nb
user: dantopa, CPU: isomer, MM v. 15.0.1 for Linux x86
date: Aug 27, 2026, time: 22:33:56
nb: /home/dantopa/Mathematica_files/nb/projects/bae/hyperion/simulation/confidence/
compute-ave-sd-01.nb
```

modules, functions, settings, ...

1 access db

```
In[46]:= (* open db connection *)
dbFile = dirData <> "confidence.sqlite";

In[47]:= (*open fresh database*)
db = OpenSQLConnection[JDBC["SQLite", dbFile]];

In[48]:= scores = SQLExecute[db, "
    SELECT overall, assess_a, assess_b, assess_c
    FROM confidence
    ORDER BY form_number
    "];
```

```
In[49]:= results = SQLExecute[db, "
      SELECT wbs_primitive, overall, assess_a, assess_b, assess_c
      FROM confidence
      ORDER BY form_number
      "];
```

```
In[50]:= CloseSQLConnection[db];
```

2

```
In[51]:= Length[results]
```

```
Out[51]=
1095
```

```
In[52]:= results[[1]]
```

```
Out[52]=
{1.1.1.1, 0, 1, 1, 1}
```

```
In[60]:= wbs = DeleteDuplicates[results[[All, 1]]]
nTypes = Length[wbs]
```

```
Out[60]=
{1.1.1.1, 1.1.1.2, 1.1.1.3, 1.2.1, 1.4, 1.2.3, 1.2.4,
 1.1.2, 1.1.3, 1.1.4, 1.1.5, 1.1.6, 1.1.7, 1.3, 1.5, 1.6, 1.7, 1.8}
```

```
Out[61]=
18
```

```
(*Method 1:Using GroupBy to create matrices*)
groupedData = GroupBy[results, First → Rest];
matrices = GroupBy[results, First → Rest, Transpose[{{#}}] &];

keys = Keys[groupedData];
```

```
Out[139]=
{1.1.1.1, 1.1.1.2, 1.1.1.3, 1.2.1, 1.4, 1.2.3, 1.2.4,
 1.1.2, 1.1.3, 1.1.4, 1.1.5, 1.1.6, 1.1.7, 1.3, 1.5, 1.6, 1.7, 1.8}
```

```
In[59]:= matrices[[2]]
```

```
Out[59]=
```

```
{{{0, -1, -1, -1}}, {{0, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}},
  {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, 1}}, {{-1, -1, 1, -1}}, {{-1, -1, -1, -1}},
  {{-1, -1, -1, 1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}},
  {{-1, -1, -1, -1}}, {{-1, -1, -1, 1}}, {{0, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}},
  {{-1, -1, -1, -1}}, {{-1, 1, -1, -1}}, {{0, -1, -1, -1}}, {{-1, -1, 0, -1}}, {{-1, -1, 1, -1}},
  {{-1, -1, 0, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, 1}}, {{-1, -1, -1, -1}}, {{0, -1, -1, -1}},
  {{-1, -1, -1, -1}}, {{-1, 0, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, 0, -1, -1}}, {{-1, -1, -1, -1}},
  {{0, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{1, -1, -1, 1}}, {{-1, -1, -1, -1}},
  {{-1, -1, -1, -1}}, {{-1, 1, 1, -1}}, {{-1, -1, 1, -1}}, {{-1, -1, 1, -1}}, {{1, -1, -1, -1}},
  {{1, -1, -1, -1}}, {{-1, -1, 0, -1}}, {{-1, 0, 0, -1}}, {{-1, -1, -1, -1}}, {{1, -1, -1, -1}},
  {{0, -1, -1, -1}}, {{-1, -1, 1, -1}}, {{-1, -1, -1, 1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}},
  {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, 1}},
  {{-1, -1, -1, -1}}, {{-1, 0, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}},
  {{-1, 0, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{1, 0, -1, -1}}, {{-1, -1, -1, -1}},
  {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}},
  {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, 1, -1, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}},
  {{1, -1, 0, -1}}, {{-1, -1, -1, -1}}, {{-1, -1, -1, -1}}, {{1, -1, -1, -1}}, {{1, -1, 0, -1}}}
```

```
In[64]:= matrix = Flatten[matrices[[2], 1]
```

```
Out[64]=
```

```
{{0, -1, -1, -1}, {0, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1},
  {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, 1}, {-1, -1, 1, -1}, {-1, -1, -1, -1},
  {-1, -1, -1, 1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1},
  {-1, -1, -1, -1}, {-1, -1, -1, 1}, {0, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1},
  {-1, -1, -1, -1}, {-1, 1, -1, -1}, {0, -1, -1, -1}, {-1, -1, 0, -1}, {-1, -1, 1, -1},
  {-1, -1, 0, -1}, {-1, -1, -1, -1}, {-1, -1, -1, 1}, {-1, -1, -1, -1}, {0, -1, -1, -1},
  {-1, -1, -1, -1}, {-1, 0, -1, -1}, {-1, -1, -1, -1}, {-1, 0, -1, -1}, {-1, -1, -1, -1},
  {0, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {1, -1, -1, 1}, {-1, -1, -1, -1},
  {-1, -1, -1, -1}, {-1, 1, 1, -1}, {-1, -1, 1, -1}, {-1, -1, 1, -1}, {1, -1, -1, -1},
  {1, -1, -1, -1}, {-1, -1, 0, -1}, {-1, 0, 0, -1}, {-1, -1, -1, -1}, {1, -1, -1, -1},
  {0, -1, -1, -1}, {-1, -1, 1, -1}, {-1, -1, -1, 1}, {-1, -1, -1, -1}, {-1, -1, -1, -1},
  {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, 1},
  {-1, -1, -1, -1}, {-1, 0, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1},
  {-1, 0, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {1, 0, -1, -1}, {-1, -1, -1, -1},
  {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1},
  {-1, -1, -1, -1}, {-1, -1, -1, -1}, {-1, 1, -1, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1},
  {1, -1, 0, -1}, {-1, -1, -1, -1}, {-1, -1, -1, -1}, {1, -1, -1, -1}, {1, -1, 0, -1}}
```

```
In[83]:= c0 = matrix[[All, 1];
```

```
c1 = matrix[[All, 2];
```

```
c2 = matrix[[All, 3];
```

```
c3 = matrix[[All, 4];
```

```
one = c0 - c0 + 1;
```

In[100]:=

$$\mu_0 = \frac{\text{one} \cdot c_0}{\text{one} \cdot \text{one}} ;$$

$$\sigma_0 = \sqrt{\frac{c_0 \cdot c_0}{\text{one} \cdot \text{one}} - \left(\frac{\text{one} \cdot c_0}{\text{one} \cdot \text{one}} \right)^2} ;$$

Out[101]=

-0.72619

Out[103]=

0.62395

In[131]:=

$$c_{\text{star}} = \frac{c_1 + c_2 + c_3}{3} ;$$

$$\mu_{\text{star}} = \frac{\text{one} \cdot c_{\text{star}}}{\text{one} \cdot \text{one}} ;$$

$$\sigma_{\text{star}} = \sqrt{\frac{c_{\text{star}} \cdot c_{\text{star}}}{\text{one} \cdot \text{one}} - \left(\frac{\text{one} \cdot c_{\text{star}}}{\text{one} \cdot \text{one}} \right)^2} ;$$

Out[133]=

-0.825397

Out[135]=

0.288566

In[138]:=

groupedData[[1]]

Out[138]=

```
{{0, 1, 1, 1}, {0, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1},
{1, 1, 1, -1}, {1, 1, -1, 1}, {1, 1, 1, 1}, {1, 1, 1, -1}, {1, 1, 1, 1}, {1, 1, 1, 1},
{1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, -1}, {0, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1},
{1, 1, 1, 1}, {1, -1, 1, 1}, {0, 1, 1, 1}, {1, 1, 0, 1}, {1, 1, -1, 1}, {1, 1, 0, 1},
{1, 1, 1, 1}, {1, 1, 1, -1}, {1, 1, 1, 1}, {0, 1, 1, 1}, {1, 1, 1, 1}, {1, 0, 1, 1}, {1, 1, 1, 1},
{1, 0, 1, 1}, {1, 1, 1, 1}, {0, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {-1, 1, 1, -1},
{1, 1, 1, 1}, {1, 1, 1, 1}, {1, -1, -1, 1}, {1, 1, -1, 1}, {1, 1, -1, 1}, {-1, 1, 1, 1},
{-1, 1, 1, 1}, {1, 1, 0, 1}, {1, 0, 0, 1}, {1, 1, 1, 1}, {-1, 1, 1, 1}, {0, 1, 1, 1},
{1, 1, -1, 1}, {1, 1, 1, -1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1},
{1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, -1}, {1, 1, 1, 1}, {1, 0, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1},
{1, 1, 1, 1}, {1, 0, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {-1, 0, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1},
{1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, -1, 1, 1}, {1, 1, 1, 1},
{1, 1, 1, 1}, {-1, 1, 0, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {-1, 1, 1, 1}, {-1, 1, 0, 1}}
```

3

In[155]:=

```
strm = OpenWrite[dirData <> "means.csv", PageWidth → ∞]
```

Out[155]=

```
OutputStream[ Name: means.csv  
Unique ID: 3]
```

In[156]:=

```

count = 0;
Do[
  wbs = keys[[k]];
  (* harvest matrices *)
  matrix = Flatten[matrices[[k], 1];
  (* extract vectors *)
  (* overall *)
  c0 = matrix[[All, 1]];
  one = c0 - c0 + 1;
  m = one.one;
  count = count + m;
  (* mean *)

$$\mu_0 = \frac{\text{one.c0}}{m} // N;$$

  (* standard deviation *)

$$\sigma_0 = \sqrt{\frac{c0.c0}{m} - \left(\frac{\text{one.c0}}{m}\right)^2} // N;$$

  (* elementals *)
  c1 = matrix[[All, 2]];
  c2 = matrix[[All, 3]];
  c3 = matrix[[All, 4]];
  (* form cstar *)

$$cstar = \frac{c1 + c2 + c3}{3};$$


$$\mu_{star} = \frac{\text{one.cstar}}{m} // N;$$

  (* standard deviation *)

$$\sigma_{star} = \sqrt{\frac{cstar.cstar}{m} - \left(\frac{\text{one.cstar}}{m}\right)^2} // N;$$

  (* output data *)
  WriteString[strm, ExportString[{{wbs,  $\mu_0$ ,  $\sigma_0$ ,  $\mu_{star}$ ,  $\sigma_{star}$ }}, "CSV"]];
  , {k, nTypes}];
Print[count, " records processed"]
1095 records processed

```

In[159]:=

Close[strm]

Out[159]=

```

/home/dantopa/Mathematica_files/io/projects/bae/hyperion/simulation/confidence/data/means.
csv

```

roll ups

develop

In[217]:=

```
"1.1.1.1", "1.1.1.2", "1.1.1.3", "1.1.1"
```

In[217]:=

```
strm = OpenWrite[dirData <> "roll-ups.csv", PageWidth → ∞]
```

Out[217]=

```
OutputStream[ Name: roll-ups.csv  
Unique ID: 3]
```

In[218]:=

```
count = 0;
coverall = {};
items = {};
{ $\alpha$ ,  $\omega$ } = {1, 3};
Do[
  (*tags *)
  wbs = keys[[k]];
  items = StringJoin[items, wbs];
  If[k <  $\omega$ , items = StringJoin[items, ", "]];
  (* harvest matrices *)
  matrix = Flatten[matrices[[k]], 1];
  (* overall scores *)
  c0 = matrix[[All, 1]];
  (* census *)
  one = c0 - c0 + 1;
  Print["{k, m} = ", {k, m}];
  m = one.one;
  count = count + m;
  (* assemble vector *)
  coverall = Join[coverall, c0]
  , {k,  $\alpha$ ,  $\omega$ ];
Print[count, " records processed"]

{k, m} = {1, 94}
{k, m} = {2, 84}
{k, m} = {3, 84}

262 records processed
```

In[224]:=

Close[strm]

Out[224]=

/home/dantopa/Mathematica_files/io/projects/bae/hyperion/simulation/confidence/data/roll-ups
.csv

test

In[291]:=

```

count = 0;
coverall = {};
onebig = {};
c1big = {};
c2big = {};
c3big = {};
items = {};
{ $\alpha$ ,  $\omega$ } = {1, 3};
rollup = "1.1.1";
Do[
  (*tags *)
  wbs = keys[[k]];
  items = StringJoin[items, wbs];
  If[k <  $\omega$ , items = StringJoin[items, ", "]];
  (* harvest matrices *)
  matrix = Flatten[matrices[[k]], 1];
  (* overall scores *)
  c0 = matrix[[All, 1]];
  (* census *)
  one = c0 - c0 + 1;
  onebig = Join[onebig, one];
  Print["{k, m} = ", {k, m}];
  m = one.one;
  count = count + m;
  (* assemble vector *)
  coverall = Join[coverall, c0];
  (* elemental scores *)
  {c1, c2, c3} = Transpose[matrix[[All, 2 ;; 4]]];
  c1big = Join[c1big, c1];
  c2big = Join[c2big, c2];
  c3big = Join[c3big, c3];
  cstar =  $\frac{c1big + c2big + c3big}{3}$ ;
  , {k,  $\alpha$ ,  $\omega$ };
Print[count, " records processed"];
(* mean *)

```

```


$$\mu_0 = \frac{\text{onebig.coverall}}{\text{onebig.onebig}} // N;$$

(* standard deviation *)

$$\sigma_0 = \sqrt{\frac{\text{coverall.coverall}}{\text{onebig.onebig}} - \left(\frac{\text{onebig.coverall}}{\text{onebig.onebig}}\right)^2} // N;$$


$$\mu_{\text{star}} = \frac{\text{onebig.cstar}}{\text{onebig.onebig}} // N;$$

(* standard deviation *)

$$\sigma_{\text{star}} = \sqrt{\frac{\text{cstar.cstar}}{\text{onebig.onebig}} - \left(\frac{\text{onebig.cstar}}{\text{onebig.onebig}}\right)^2} // N;$$

Print[rollup, ":  $\mu_0 \pm \sigma_0 =$ ",  $\mu_0$ , "+/-",  $\sigma_0$ ]

{k, m} = {1, 94}
{k, m} = {2, 84}
{k, m} = {3, 84}
262 records processed
1.1.1:  $\mu_0 \pm \sigma_0 = 0. \pm 0.766672$ 

```

end